

1 Lesson 23-09-26

1.1 Processor design

The first general-purpose computer was created in the late 50s. A Personal Computer, that can now be bought for about \$500, is practically equivalent (in terms of performance and memory) to what could be bought for about \$1M in 1985.

This evolution has been made possible by:

- Advances in semiconductor technology
- Innovations in computer architecture
- Advances in software.

Despite this the main architecture used today is still the same that was first applied to real computers, that is the *Von Neumann Architecture*.

1.1.1 Advanced architectures

Of course, there are modern advancements which have to be noted. We refer to these as *advanced architectures*:

- **Advances in computer architecture:** performance has been improved without fundamentally changing the Von Neumann model through techniques such as pipelining, which overlaps the execution of multiple instructions, and superscalar execution, which allows multiple instructions to be executed in parallel in a single clock cycle. Other major improvements include caches, branch prediction, and out-of-order execution.
- **Advances in software and tooling:** compilers have become increasingly sophisticated at optimizing programs for the underlying hardware, exploiting instruction-level parallelism, registers, caches, and specialized instructions. Better operating systems, programming languages, debuggers, profilers, and development tools have also made increasingly complex hardware practical to use.

Let's now focus on the processor component of most computer systems.

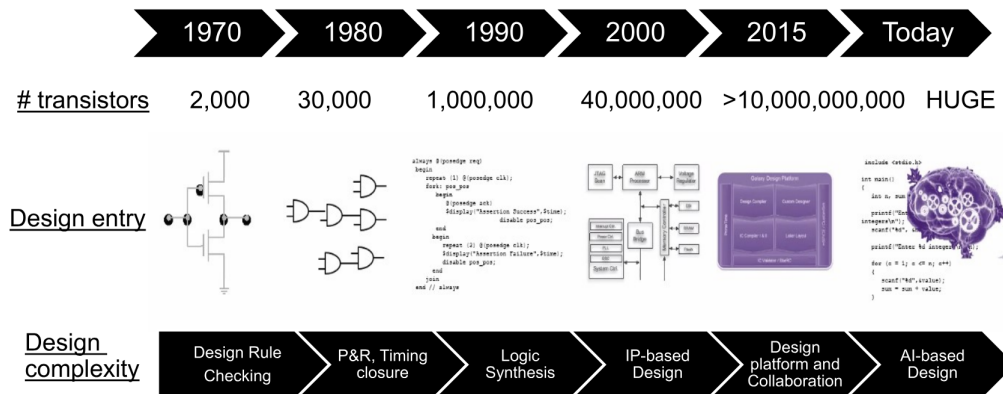
1.1.2 Moore's law

Definition 1.1: Moore's law

The number of devices (i.e., transistors) that can be integrated into a single chip doubles every 18/24 months.

Thanks to Moore's law, we are today able to integrate more and more transistors within one circuit, giving many powerful options to designers.

Chip-design history



Many technologies are available to streamline chip design (for instance, *synthesis* and automated tools). Having more transistors within a single processor also allows for advanced features. For instance, something like a *floating point* processing unity executing alongside the main processor takes a large number of transistors to implement, which larger chip transistor densities allow.

1.1.3 Costs

Costs have to be taken into account. There are 2 main types of costs:

- **Design costs:** they are largely *fixed*/non-recurring costs (**NRE**) and grow rapidly with chip complexity and process-node sophistication;
- **Manufacturing costs:** we note that manufacturing costs are dominated by the cost for building a new fab, which explodes when moving to advanced semiconductor nodes.

We also note that when evaluating manufacturing costs, it is important not to forget the impact of yield, i.e., the percentage of products that pass the test phase. The production process for every product undergoes an evolution which normally leads to an improvement in yield (also known as learning curve). When yield increases, the cost decreases. More than 50% of manufacturing cost is due to validation and testing procedures!

1.1.4 Processor performance evolution

Processor **performance** can be measured by counting the number of operations executed or the time needed to execute them.

- User point of view: performance = response time (time between start and completion of an operation);
- System manager point of view: performance = *throughput* (total amount of work done in the time unit).

Processor performance evaluation is often performed by letting the computer to execute applications and observing its behavior. Unfortunately, the choice of the applications severely affects the performance. In the ideal case, one should use as workload the mix of applications the user will run. However, they are normally unknown, and largely variable from one user to another. Therefore, some **benchmarks** are selected to mimic real cases.

Processor performance has steadily increased since the 1970s, where the first main-frame machines started being ranked based on performance.

- From the 1970s to around the 1980s, yearly performance increases were of about 25%;
- In the 1980s, the introduction of **RISC** (*Reduced Instruction Set Computer*) architectures, as opposite to **CISC**, led to a yearly performance gain of around 50%;
- In the first years of the 2000s, processor performance growth hit a slow down because aiming for higher and higher clock speeds became unfeasible (mainly because of power dissipation constraints). This was disadvantageous for the *superscalar* architectures used at the time, characterized by high pipeline depth and instruction level parallelism;
- The solution most vendors adopted from the mid 2000s onwards, and which to this day is widely adopted, is *multi-core* processors. In a *multi-core* processor, we effectively have multiple processors (or processor *cores*) running in parallel on a single, or multiple tasks. Note that multi-core setups increase *throughput*, and not *response time*.

Today, we can talk about the fact that modern computer systems employ various types of *accelerators*, that is circuits intended to support specific applications alongside the CPU, like graphics or AI.

In the recent years, the widespread adoption of Artificial Intelligence (AI) applications prompted the introduction of new computing architectures denoted as AI accelerators, such as **GPUs** (*Graphics Processing Units*). GPUS were originally introduced for graphics applications, but their current success is due to their ability to effectively manage large and regular data structures. The GPU market (in 2024 corresponding to about 18 B\$), is expected to grow to 406 B\$ by 2033 (with a Compound Average Grow Rate, or CAGR, of 41,39%). The CAGR of the CPU market is about 7%.

1.1.5 The computer market

The computer *market* is currently split in 5 different areas:

- *Personal Mobile Devices (PMDs)*: this area includes smartphones and tablet computers. Emphasis is on energy efficiency and real-time applications. Here, system price is around \$100 - \$1,000, with the microprocessor price at \$10 - \$100. Architectures are usually highly parallel (and in fact smartphones were the earliest devices to make heavy use of multi-core processors);
- Desktop computing: this area covers from PCs to workstations (and in some sense, even laptops). The main target of this area is to optimize the price-performance ratio. The system price is around \$300 - \$2,500, and of course the microprocessor price follows \$50 - \$500. Power is much less important than before, as we don't use batteries but power off the grid;

- Servers: these systems provide larger-scale and more reliable computing services. The main parameters of products in this area are:

- Availability;
- Scalability;
- Throughput.

System price is around \$5,000 - \$10,000,000, with microprocessor price around \$200 - \$2,000;

- Clusters / *Warehouse Scale Computers (WSC)*: systems that offer “Software as a Service (*SaaS*)” or “Platform as a Service (*PaaS*)”. The emphasis is on availability, price-performance, and power consumption. *Supercomputers* make up a part of this category, where the emphasis is on floating-point performance and fast internal networks. The system price is high at around \$100,000 - \$200,000,000, with the microprocessor price around \$50 - \$250 (because at times we might use off the shelf processors in massively parallel setups);
- Embedded computers: this area is a major application area of the computer market. It covers all special-purpose computer-based applications (from microwaves to coffee machines, from automotive to videogames). Adopted microprocessors vary from cheap low-end 8-bit processors to very efficient (and expensive) high-end processors. The system price ranges from \$10 - \$100,000, with microprocessor price at around \$0.01 - \$100 (possibly extremely cheap).

Special requirements often existing in this area are:

- Real-time performance requirements;
- Memory minimization;
- Power consumption minimization;
- Reliability constraints.

Embedded problems are often solved resorting to one of the following "standard" solutions:

- Standard processor, custom logic, custom software. We call the combination of standard processors with custom logic **ASIC** (*Application Specific Integrated Circuit*);
- Standard processor, custom software;
- Standard DSP, custom software.

Programmable devices (such as **FPGAs**) are playing a growing role.

1.1.6 Power

Continuous increase in system complexity and device integration often leads to problems with **power consumption**. Power consumption may be critical under two aspects:

- **Power**: *static* power, consumed when a device is just running without doing work, and *dynamic* power, which mainly depends on the frequency devices run at (and also the square power). i Until now it has been dominated by dynamic power, i.e.,

that consumed by each transistor when switching between different states. The dynamic power for each transistor can be evaluated by the usual formula:

$$P_{dynamic} = fV^2C_{eq}$$

whereas static power can be evaluated by:

$$P_{static} = VI$$

For this reason, voltage continuously dropped in the last years.

- **Energy:** it is given by:

$$E = C_{eq}V^2$$

It is mainly of interest for mobile devices.

1.1.7 Dependability

Dependability is the quality of the system to deliver a correct service. Dependability of computer systems is traditionally very high, but it can be lowered by:

- Bugs in the design of the hardware;
- Bugs in the software;
- Defects in the hardware (introduced by the manufacturing process);
- Faults happening during the product operation.

Many **safety critical** systems exist. In the past, possible misbehaviors of computer systems were critical in some areas, such as

- Space;
- Avionics;
- Nuclear plants control.

In more recent years, computer-based systems expanded to other safety-critical areas, such as:

- Rail-road traffic control
- Automotive
- Biomedical
- Telecommunications.

Dependability *metrics* exist. Dependability is often measured using:

- *Mean Time To Failure (MTTF)* or *Failures In Time (FIT)*, which is the reciprocal. 1 FIT equals 1 failure in one billion hours;
- *Mean Time To Repair (MTTR)*;
- *Mean Time Between Failures (MTBF)*.

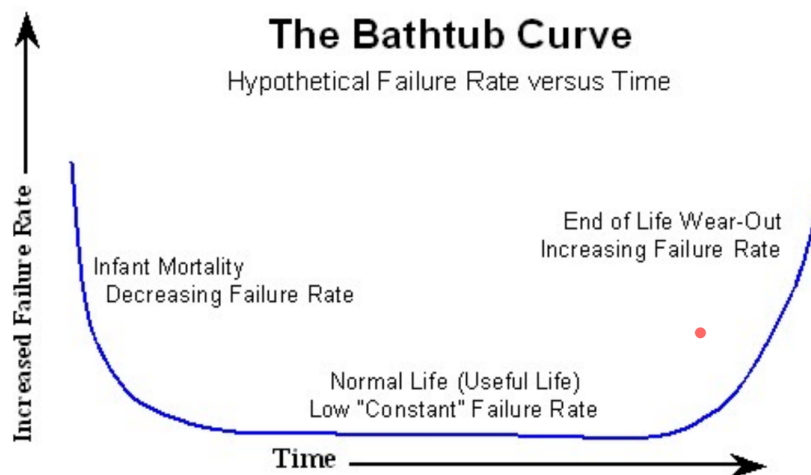
The three measures are related by the formula:

$$MTBF = MTTF + MTTR$$

Finally, availability is the probability that a system works correctly in a generic time instant (usually measured in FITs).

We provide a figure of the typical *bathub curve* that describes the probability of faults given the age of a component.

- The *infant mortality* zone is where faults of components show themselves early. Controlled *burn-in* can partially solve this problem;
- The *normal life* zone is where expect to spend time using devices;
- The *end of life* zone is where failures hidden in components which otherwise worked fine start appearing. This is of particular interest to data centers where components are heavily stressed.



1.1.8 Reproducibility

Going back to the topic of *performance*, we introduce **reproducibility**. Information about execution times on benchmarks should allow reproducibility. This means reporting detailed information about:

- Hardware (system configuration);
- Software (OS, compiler, program);
- Program input.

A common issue is comparing and summarizing performance:

- **Problem 1:** I know the performance of one machine on a set of programs: which is its global performance?
- **Problem 2:** I know the performance of two machines on the same set of programs: which is their relative performance?

A number of metrics have been proposed, such as total execution time, arithmetic mean and weighted means.

1.1.9 Ahmdal's law

Definition 1.2: Ahmdal's law

We define the *speedup* from a given architecture enhancement as:

$$\text{speedup} = \frac{\text{performance with enhancement}}{\text{performance without enhancement}}$$

The speedup resulting from an enhancement depends on two factors:

- $\text{fraction}_{\text{enhanced}}$: the fraction of the computation time that takes advantage of the enhancement;
- $\text{speedup}_{\text{enhanced}}$: the size of the enhancement on the parts it affects.

We can compute the execution time after the speedup as:

$$\text{execution time}_{\text{new}} = \text{execution time}_{\text{old}} \cdot ()$$

Applying the definition of speedup, we get:

$$\text{speedup} = \frac{\text{performance with enhancement}}{\text{performance without enhancement}} = \frac{1}{(1 - \text{fraction}_{\text{enhanced}}) + \frac{\text{fraction}_{\text{enhanced}}}{\text{speedup}_{\text{enhanced}}}}$$

Ahmdal's law is particularly useful in the study of parallel architectures, as it expresses the speedup given by optimizing parallelizable parts of computing tasks. More specifically, we use it to explain why we can never have $S = N$ with N processor cores.

We can say that there will always be intervals of sequential execution (T_{seq}), spent executing instructions that cannot be parallelized, such as I/O operations, conditional statements, inherently sequential algorithms, etc... Then the speedup S can be defined as:

$$S = \frac{T_1}{T_{\text{seq}} + \frac{T_1 - T_{\text{seq}}}{N}}$$

with T_1 being the full execution time, i.e., the speed-up gain comes only from the parallelizable parts of the program ($T_1 - T_{\text{seq}}$), while the sequential parts (T_{seq}) cannot benefit significantly from parallelism.

The problem is that, taking the limit for $N \rightarrow \infty$, we obtain:

$$\lim_{N \rightarrow \infty} S = \lim_{N \rightarrow \infty} \frac{T_1}{T_{\text{seq}} + \frac{T_1 - T_{\text{seq}}}{N}} = \frac{T_1}{T_{\text{seq}}}$$

Thus, as N increases, the speed-up becomes dominated by the sequential parts: the parallelizable portions are progressively reduced, while the sequential portions remain and ultimately become the limiting factor for the overall system performance.

1.1.10 The CPU performance equation

Let's talk about measuring the time required to execute a program. There are several possible approaches:

- By observing the real system;

- By simulation;
- By applying the **CPU performance equation**.

Definition 1.3: CPU performance equation

CPU execution time is given by:

$$\text{CPU Time} = \left(\sum_i \text{CPI}_i \times \text{IC}_i \right) \times T_C$$

where:

- CPI_i is the number of clock cycles required by instruction i ;
- IC_i is the number of times instruction i is executed in the program;
- T_C is the clock period.

This equation is quite basic, as it doesn't take any processor optimizations into account. Still, it is a feasible metric for processor performance estimation, before the processor is even actually implemented.

1.2 Instruction Set Principles

Every processor owns an *Instruction Set Architecture (ISA)*. The ISA provides the information about the processor required by programmers and compilers to produce correctly working code:

- The set of available *instructions*;
- Instruction *encodings*;
- Available *registers*;
- etc...

Different processors may share the same ISA (e.g., Intel and AMD processors, or processors of the same family).

The Instruction Set Architecture (ISA) is basically how the computer is seen by the programmer or the compiler. There are many alternatives possible for the ISA designer, any choice modifies the way to encode the instructions. The different alternatives may be evaluated in terms of:

- Processor performance;
- Processor complexity;
- Compiler complexity;
- Code size;
- Power consumption;
- etc...

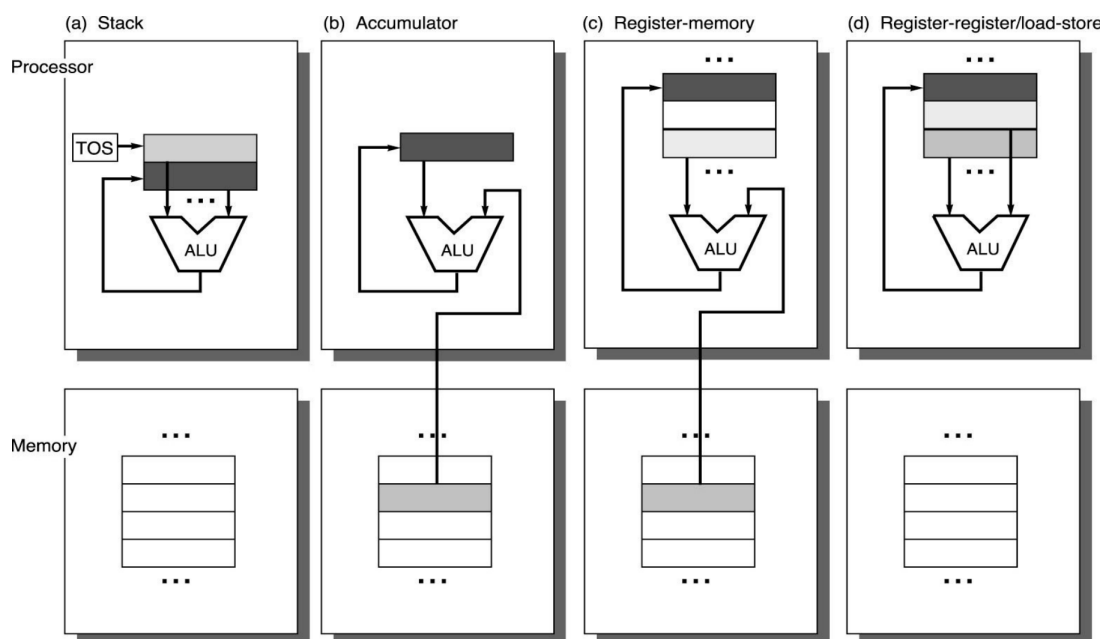
Different product areas may assign different weights to the above parameters.

An ISA specification is particularly important because it is usually the only set of information released by a company about a processor. Of course, the internal implementation details are trade secrets, but the ISA specification is made available in order to allow programmers to develop for a certain platform.

1.2.1 CPU taxonomies

CPUs (and their ISAs) are often classified according to the type of their internal storage:

- Stack;
- Accumulator;
- Registers:
 - Register-memory, as in x86;
 - Register-register (load-store). Basically all RISC architectures are of this kind;
 - Memory-memory (no real cases).



Currently, all processors are *General-Purpose Register (GPR)* machines. The reason for this is:

- Registers are faster than memory;
- Registers are easier for a compiler to use (or better, to *optimize*).

CPUs can also be classified according to:

- Typical number of operands per ALU instruction (2 or 3). In RISC-V and ARM, this is almost always 2;
- Typical number of memory operands per ALU instruction (from 1 to 3, for architectures that feature memory operands in the first place in instructions rather than load/store).

1.2.2 Memory addressing

We have some orthogonal characteristics relating to memory *addressing*:

- **Little Endian vs. Big Endian:** processors usually address memory in bytes, organized into words. The order of the bytes within a word matters:
 - *Little Endian* puts the byte with the lowest address (X...X000) at the least significant position. The address of the data is that of the least significant byte;
 - *Big Endian* puts the byte with the lowest address (X...X000) at the most significant position. The address of the data is that of the most significant byte.
- **Aligned vs. misaligned accesses:**
 - *Aligned* means allowing only aligned accesses not straddling word (usually 2 byte) bounds. It is a limitation;
 - *Disaligned* access, on the other hand, requires: hardware overhead and performance overhead.

We shall also note that in GPR machines an addressing mode specifies a constant, a register, or a memory location (through its effective address).

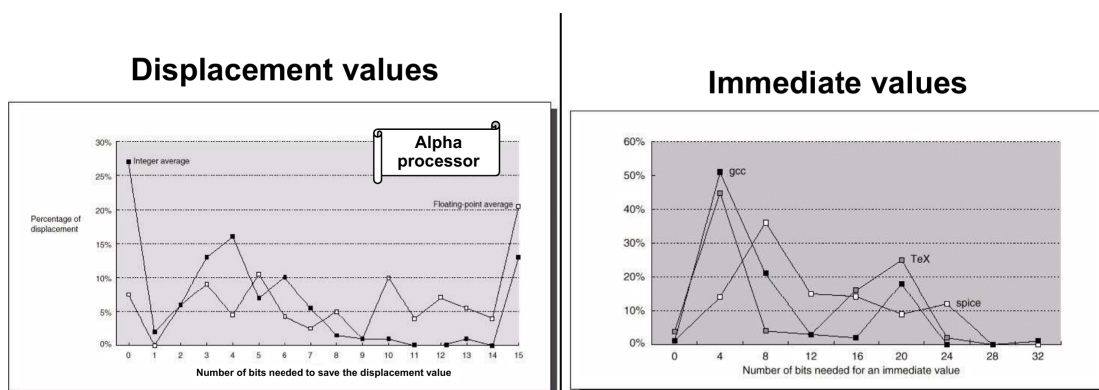
We remember the addressing modes noted in section 1.2.5 for x86. Some of these are shared with ARM and RISC-V RISC architectures (mainly register mode, with destination register, immediate and displacement modes for load/store operations).

By carefully choosing the addressing modes, one can obtain some important consequences:

- Reducing the number of instructions;
- Increasing the CPU architecture complexity;
- Increasing the average *Cycles Per Instruction (CPI)*.

An interesting thing to note is the size devoted to immediate operands in immediate instructions. That is, how large can the immediate value be, when it is embedded in the addressing part of the instruction? Note that memory displacements basically are a special case of immediate operands, and as such fall in the same category and have the same problem. In fact, we had already noted this issue in section 1.2.5.

An experimental evaluation can be performed.



From the plots, compiled from a DEC alpha processor for displacement and for typical compiled applications for immediates, we get that:

- Displacement, immediate and register indirect modes represent from 75% to 99% of the addressing modes;
- The address size for displacement mode should be from 12 to 16 bits (75% to 99% of the displacements);
- The size of the immediate field should be at least 8 or 16 bits (50% and 80% of the cases, respectively).

RISC-V uses the solution of doing 12 bits immediates. As we previously stated, bigger immediates have to be transferred to register in multiple instructions. This tradeoff is made valid by the simplicity gains of the processor architecture.

1.3 Operations in the instruction set

Clearly, a key component of an ISA is the **instruction set** made available to the programmer. Different categories of instructions exist:

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, and, subtract, or
Data transfer	Loads-stores (move instructions on machines with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel operations, compression/decompression operations

We should be making the common case fast: not all the instructions are executed with the same frequency! When designing and implementing an instruction set, the most commonly executed instructions should be made faster than others.